Adding a new modality to the reader study

A step-by-step protocol, as run for the chest-CT and chest-X-ray arms, using the reader-study app at github.com/jomoll/annotation_app. Written for a collaborator setting up a new arm from scratch.

0. Know what the candidates are for

The study validates an automatic report-quality **metric** against clinician judgement. It does not benchmark models. That single fact decides most of the design questions below: candidate reports are **stimuli**, not subjects. What the stimulus set must do is span the quality range and cover the error types the metric has to detect. Which generator produced a given report matters only as a means to that end.

Two consequences people get wrong. You do not need the candidate models to be separable from one another, so do not add models hoping to tell them apart. And if the models available for your modality genuinely cannot span the range, the candidates do not all have to come from models at all.

1. Choose the dataset and fix the split

- Use a **held-out split only** (test, or validation if the models were tuned on test). Candidate models must not have trained on the studies your readers see.
- Check the licence before anything else. Gated or non-commercial datasets constrain where images may be sent, which matters if you later call a hosted model.
- Confirm the reference reports have usable sections. We rely on **findings** and **impression**; clinical information and technique are shown for context but are not scored.

> **Verify the split actually contains what you need**
> For the MRI arm the dataset was advertised as brain and spine, but the test split proved to be 97.8% brain, 2.2% combined brain-and-spine, and exactly one spine study. A planned spine stratum was impossible. Count your strata in the held-out split before designing around them.

2. Select and filter studies

1. **One study per patient.** Keep the earliest; a follow-up leaks information even when its own report looks self-contained.
2. **Exclude reports that reference a prior.** Models see one study and cannot know what changed. Use a strong-phrase regex for the obvious cases ("compared with", "previous examination", a de-identified date token) and send borderline wording ("stable", "unchanged", "interval") to an LLM to decide. Log every exclusion with its reason.
3. **Stratify** on whatever drives rating variance in your modality, and cap the normal share.
4. **Over-select** by roughly 3x so you can drop studies later without re-sampling.

> **Cap the normal share deliberately**
> In our X-ray arm, normal-versus-abnormal explained 49% of the variance in overall scores, against 4.7% for model identity. Readers rate normal studies highly almost regardless of candidate quality. We rebuilt the pool at a 15% normal share. Decide this before generating candidates, not after.

3. Generate candidate reports

The default design is four or five models specialised for the modality, and nothing else. That is what both existing arms do, and a new arm should match them unless it is forced not to. Specialised models fail in the ways real systems fail, which is what the metric has to cope with, and keeping the arms alike keeps them

comparable.

One candidate per reference. Each study gets exactly one candidate report, and the systems are spread equally at random within each stratum. Our arms are 200 distinct studies with five systems at 40 each. Showing the same study to every model instead spends the annotation budget on correlated judgements of one reference; with a fixed budget, prefer reference diversity.

Released outputs count. Four of our five chest-CT systems are the authors' own published predictions rather than anything we ran, and one X-ray system likewise. Look for released test-split outputs before building an inference environment.

Fall back only if step 4 says the pool is inadequate

In ascending order of distance from the default. Add the least you need, and record which strategy produced each candidate so it can be a stratum in the analysis.

Fallback	What it gives you	Reach for it when
More specialised models	Same failure modes, wider spread	Any remain untried for the modality
General-purpose model on rendered slices	Fluent, confident, often wrong	Specialised models are few or weak
Controlled error injection	Errors known by category, span and severity	The top of the range is empty
Retrieval of a similar case	Fluent, correctly styled, plausibly wrong	A principled floor is missing

The last two are implemented in this repository and need no other install: `scripts/generate_fallbacks.py` (lexicons for ultrasound or any other modality go in a JSON file, see `reporttools/lexicons/`). Both derive from the reference text, so keep them as their own stratum in every analysis.

```
python3 scripts/generate_fallbacks.py coverage --modality <m> --acquisition <a>
python3 scripts/generate_fallbacks.py inject   --reports refs.csv --studies eval.txt \
        --modality <m> --acquisition <a> --out data/raw/<m>/candidates/injected.jsonl
python3 scripts/generate_fallbacks.py retrieve --reports refs.csv --studies eval.txt \
        --donor-reports train.csv --out data/raw/<m>/candidates/retrieved.jsonl
```

On brain MRI a general-purpose model beat every specialised one we tried, so the ordering above is about staying close to the other arms, not about which model is best. Three brain-MRI models bear this out: the best reached 0.31 median F1 against 0.36 for a general-purpose model.

> **A low claim-F1 can be the metric's fault, not the model's**
> One brain-MRI model scored 0.09 and looked like a total failure. It was not. On the studies it summarised as “study is unremarkable” the reference was genuinely normal and the judge marked its single claim CORRECT — but the reference enumerates each normal structure separately, so a normal study parses into 19-29 normality claims against the candidate's 1. F1 = 2x1/(19+1) = 0.10 for a report that is right. Claim-level F1 systematically punishes correct, concise normality reporting, and the penalty scales with how verbosely the reference lists normal structures. Before dropping a model on a low median F1, check the normal studies separately: look at correct/candidate_claims, not F1 alone. The same caution applies to any arm whose references enumerate normality, which is most CT and MR reporting.

Record for every candidate: study id, system, strategy, findings, impression, which image series were used, and a prompt version string. You will want all of it when a result looks odd.

4. Check the spread before booking any clinician time

Score every candidate against its reference with an automatic metric and look at the **distribution**, not the mean. This is the step that saves a wasted study.

Any claim-level metric will do for this check — for example RadMatch (github.com/raidium-med/RadMatch) or radmetric (github.com/jomoll/radmetric). Score one row per candidate and keep the per-report output.

Arm	Median F1	Reports with no correct claim	Clearing 0.3
Chest CT	0.46	0%	92%
Chest X-ray	0.33	8%	65%
Brain MRI	0.23	20%	40%

CT and X-ray span a usable range with specialised models alone and need no fallback. Brain MRI does not reach the top of the scale, which is the only reason that arm supplements its models.

The decision rule. If the pool has no empty bin and a few per cent of candidates clear the top of the scale, stop: four or five specialised models is the arm, exactly as in CT and X-ray. Add a fallback strategy only to fill a bin the models leave empty, and add the smallest number that fills it. A pool whose median sits near the floor, or where a large share of candidates contain no correct claim at all, cannot be fixed by adding more of the same kind of model.

Pick a judge that is not also a candidate
Every model we had served locally was also in the candidate set, so it would have graded its own reports. We used a text-only model instead: the judge reads reference and candidate text, never images, so a text-only model qualifies and cannot be one of the candidates.

Disable thinking mode on a reasoning judge
A reasoning model returns its trace in the message body. Our claim extractor then read 4 claims out of reports holding 29, and the parser silently scored near zero. Send chat_template_kwargs {enable_thinking: false}, or use a model that returns reasoning in a separate field, and always print the parse rate. Not every server honours the switch: behind a LiteLLM gateway glm-5.3-flash kept thinking and returned empty answers once its budget ran out, so raise the answer budget instead (ANNOTATION_LLM_MAX_TOKENS for the bundled tools).

5. Stand up the app

The app is a small Streamlit reader study, published so collaborators can drop in their own reference and candidate pairs. The modality, the questions and the data source are all pluggable; nothing below needs a fork.

```
git clone https://github.com/jomoll/annotation_app.git && cd annotation_app
pip install -r requirements.txt
streamlit run app/app.py --server.address 127.0.0.1 --server.port 8750
```

- With no pool present it runs on a bundled example so you can see the interface before building anything.
- First start seeds a host account `admin / admin`. **Change that password before the app is reachable by anyone else.**
- Readers sign up themselves. Only a salted password hash is stored, never the password.
- Each reader's ratings append to their own CSV under `data/annotations/`; the latest row per case wins, so a reader may revise an answer and nothing is overwritten.
- Expose it over an SSH tunnel rather than binding it to a public interface.

The questionnaire lives in one file, `app/questions.py`. That is the file to edit for a different question set: each question carries its key, prompt, type and the per-score definitions the reader sees. Scales may differ in length between questions.

6. Build the pool and register the modality

1. Add the modality to `config.MODALITIES`.
2. Build the per-modality pool file, then merge all modalities into the single cases file the app serves. Case ids must be unique across pools.
3. Precompute the claim links that drive hover highlighting, if your readers want it. Ours asked for it and it is on by default.
4. Keep every pool file and all raw candidates out of version control.

```
# before generating: one system per study, normal/abnormal mix equal per system
python3 scripts/assign_systems.py --references refs.csv --studies eval.txt \
        --systems modelA modelB injected retrieved --out data/raw/<m>/assignment.csv
# after generating: references + candidate files -> the pool the app serves
python3 scripts/build_pool.py --modality <m> --references refs.csv \
        --candidates modelA=a.jsonl modelB=b.jsonl injected=... retrieved=... \
        --assignment data/raw/<m>/assignment.csv --out data/cases_<m>.json
python3 scripts/merge_pools.py <m>=data/cases_<m>.json --out data/cases.json
python3 scripts/precompute_links.py data/cases.json        # optional, needs an LLM
```

References may be sectioned (findings + impression) or one block of text, as pathology reports usually are; both are shown to the reader as written. build_pool.py copies only report text into the pool — the injection ground truth and retrieval's donor ids stay in the candidate files, out of the readers' browsers.

7. Set the reader allocation

Two readers per modality, each rating enough studies that the overlap reaches the share you want. For a 200-study pool and 20% overlap: 120 studies each, giving 240 slots and therefore 40 double-rated.

```
RATERS_PER_MODALITY = 2
STUDIES_PER_RATER   = 120      # 2 x 120 - 200 = 40 shared
TARGET_COVERAGE     = 2
OVERLAP_FIRST       = True     # shared block at the START of each list
```

> **Never reorder a reader's list once they have started**
> The app resumes a new session from the stored case id, but an open session tracks an integer position. Reordering moves the reader somewhere else in the list. One of our readers skipped the 40-case shared block twice this way. Put the shared block first at list creation and never touch the order again.

8. Onboard the readers

- Put the question definitions where the reader sees them on every case. A verbal briefing does not hold: after one in-person clarification our reader's scores corrected for about half a day and then drifted back past where they started.
- State explicitly what each question is *not* about. Our completeness question has never reached acceptable agreement in any arm (Spearman 0.11 to 0.25 across three measurements and two modalities) because readers disagree on whether it means coverage of the reference or presence of the clinically relevant information.
- Check for inverted binaries in the first few cases. One reader answered the safety question with the opposite polarity for 15 cases before anyone noticed.

9. Monitor while the study runs

Check	How	What is wrong if it fails
Binary polarity	Mean overall for Yes versus No on each binary, per reader	Reader has the question inverted
Reader offset	Mean difference on the shared block	Systematic generosity; correctable now, only statistically later
Agreement	Spearman and weighted kappa on the shared block	Question wording, if rank agreement is fine but kappa is not
Blank submissions	Rows with some answers missing	Ungenerated candidate, or the reader navigated away

Expect a persistent offset between readers even when they rank cases together. In our X-ray pair, rank agreement on overall is 0.72 while one reader sits a full point above the other on every scale.

10. Analyse

- Report arms separately. Never pool a correlation across modalities: our GREEN-versus-reader correlation was 0.44 on CT and 0.77 on X-ray, and the gap survives matching the sample size.
- Control for normal versus abnormal. It is the largest single confound in every arm we have measured.
- Quote the human ceiling alongside any metric correlation. A metric at 0.44 against readers who agree with each other at 0.73 is a different claim from one at 0.44 against readers who agree at 0.50.
- If candidates came from more than one strategy, treat strategy as a stratum. Injected and retrieved reports derive from reference text rather than images and cannot be presented as model output.

Checklist before the first reader logs in

- Held-out split confirmed, and the strata you planned actually exist in it
- One study per patient; prior-referencing reports excluded and logged
- Normal share capped deliberately
- Four or five specialised models, one candidate per reference, balanced within strata
- Spread checked on the distribution, with a judge that is not a candidate
- Fallback strategies added only where the models left a bin empty, and recorded as a stratum
- Shared block first in both readers' lists
- Question definitions visible on every case, with the common misreading named
- App running, admin password changed, readers able to sign up
- Raw candidates and pool files git-ignored